

PRÁCTICA 7

Pasos, giros y fusión WiFi + PDR

Navegación inercial (Pedestrian Dead Reckoning) · Dead reckoning · Fusión por varianza inversa

Duración: 2 sesiones de 1 h 45 · M5StickC Plus 2 · Python (análisis offline en Windows)

1. Introducción y contexto

En las prácticas anteriores construiste el pipeline de procesamiento de señal inercial: remuestreo, filtrado, detección de pasos y reconstrucción de trayectoria (bloques 4 y 5), y lo llevaste a tiempo real por MQTT (bloque 6). En esta práctica cierras el proyecto: conviertes el IMU en un SENSOR DE POSICIÓN de pleno derecho —navegación por estima peatonal, PDR— y lo FUSIONAS con la localización WiFi.

La clave es que WiFi y PDR tienen errores de naturaleza opuesta. El WiFi da una posición absoluta pero ruidosa y de baja cadencia: «salta». El PDR es suave y rápido, pero al integrar pasos y rumbo acumula deriva: sin una referencia absoluta se degrada con la distancia. Combinarlos de forma razonada —cada uno pesando según su fiabilidad— produce una trayectoria continua y estable. Es el ejemplo canónico de fusión sensorial complementaria y la puerta de entrada al filtro de Kalman.

2. Objetivos de aprendizaje

- Implementar un pipeline PDR completo sobre datos reales del IMU: remuestreo uniforme, filtrado, detección de pasos y estimación del rumbo por integración del giróscopo.
- Reconstruir la trayectoria por dead reckoning y comprender por qué la incertidumbre crece con la distancia recorrida (deriva).
- Fusionar el fix WiFi con la predicción PDR mediante ponderación por varianza inversa (filtro de Kalman escalar simplificado).
- Razonar sobre la complementariedad de dos sensores y sobre el papel de la incertidumbre en la fusión.

3. Material y ficheros de partida

Software: Python 3.10+ (Windows). Librerías: numpy, pandas, scipy, matplotlib.

```
pip install numpy pandas scipy matplotlib
```

Ficheros (carpeta datos/):

- imu_log.csv, imu_log2.csv — registros reales del IMU del M5StickC caminando por los pasillos (columnas: ts_ms, ax, ay, az, gx, gy, gz). imu_log2.csv contiene 115 pasos reales, útil para validar el contador.

Plantillas (carpeta code/):

- pdr_pipeline.py — pipeline PDR con 4 TODOs (Bloque A).
- fusion_wifi_pdr.py — fusión WiFi + PDR con 1 TODO (Bloque B).

4. Bloque A — PDR: pasos, giros y trayectoria

Trabaja sobre code/pdr_pipeline.py. El IMU del ESP32 genera timestamps irregulares (jitter): antes de cualquier filtrado hay que eliminar lecturas duplicadas y remuestrear a una tasa uniforme (50 Hz) — esto ya está resuelto en `cargar_remuestreado()`. Completa los cuatro pasos de procesamiento:

- TODO 1 — Filtrar la magnitud de aceleración con un paso-bajo Butterworth (orden 4, corte 3 Hz) para quedarte con la cadencia del paso y eliminar vibraciones.
- TODO 2 — Detectar los pasos como picos de esa señal (`find_peaks`, con prominencia y separación mínimas).

- TODO 3 — Estimar el rumbo integrando el giróscopo del eje vertical (gz en °/s → radianes).
- TODO 4 — Dead reckoning: por cada paso, avanzar la longitud de paso en la dirección del rumbo ($\Delta x = L \cdot \sin \theta$, $\Delta y = L \cdot \cos \theta$).

Verificación: con imu_log2.csv debes detectar del orden de 115 pasos y obtener una trayectoria que reproduce el recorrido (se guarda en pdr_resultado.png).

Cuestión para la memoria: el rumbo se obtiene integrando el giróscopo, así que es relativo al arranque y su error se acumula (deriva del bias). ¿Qué sensor daría un rumbo absoluto? ¿Por qué la trayectoria se «tuerce» poco a poco aunque el sujeto camine recto?

5. Bloque B — Fusión WiFi + PDR

Trabaja sobre code/fusion_wifi_pdr.py, que simula una secuencia: llega un fix WiFi cada cierto tiempo y, entre medias, el PDR predice el desplazamiento. Completa la fusión por varianza inversa: cada estimación pesa según el inverso de su varianza.

```
sigma_pdr = max(SIGMA_PDR_PASO * sqrt(n_pasos), 0.5)
w_wifi = 1 / MAE_WIFI**2
w_pdr = 1 / sigma_pdr**2
x = (w_wifi*x_wifi + w_pdr*x_pdr) / (w_wifi + w_pdr)
```

Observa el efecto: justo tras un fix WiFi, la incertidumbre del PDR es pequeña y su predicción domina, dando una trayectoria suave; a medida que se acumulan pasos sin corrección, sigma_pdr crece (con la raíz del número de pasos) y el sistema vuelve a confiar en el WiFi, que lo reancla. Compara en la figura la trayectoria SOLO-WiFi (que salta) con la FUSIONADA (continua).

Cuestión para la memoria: ¿por qué sigma_pdr crece con la raíz del número de pasos y no linealmente? ¿Qué ocurriría si nunca llegara un fix WiFi? ¿Y si subieras artificialmente la confianza en el PDR (bajar SIGMA_PDR_PASO)?

6. Conexión con el proyecto AINI (Hito 3)

Esta práctica es el motor del tercer hito del proyecto. En el sistema real, el M5StickC calcula el PDR a bordo (a 50 Hz) y envía en cada mensaje MQTT el bloque imu_pdr = {dx_m, dy_m, steps, heading}; el servidor lo fusiona con el fix WiFi del método C (bloque 6) mediante exactamente esta ponderación por varianza inversa. El resultado es el marcador que, en la demostración final, sigue el pasillo de forma suave en lugar de saltar de punto a punto.

7. Entregable

- pdr_pipeline.py completo, con pdr_resultado.png (detección de pasos + trayectoria).
- fusion_wifi_pdr.py completo, con fusion_resultado.png comparando solo-WiFi frente a fusionada.
- Memoria breve (1–2 páginas) respondiendo a las cuestiones de los bloques A y B.

8. Ampliación opcional (bonus)

- ZUPT (Zero-Velocity Update): usar los tramos «quieto» para forzar desplazamiento nulo y frenar la deriva.
- Corregir el bias del giróscopo en reposo antes de integrar el rumbo.
- Reproducir la fusión sobre una traza real (imu_log.csv) inventando fixes WiFi periódicos y midiendo cuánto mejora frente a solo-PDR.